# 76. Массивы и циклы

На этом уроке мы рассмотрим использование массивов с циклами, а также подводные камни, с которыми вы можете при этом столкнуться.

Оглавление:


1. Зачем использовать циклы с массивами?
2. Циклы и массивы
3. Использование циклов с массивами
4. Массивы и «ошибка неучтенной единицы»
5. Тест


## Зачем использовать циклы с массивами?

Рассмотрим случай, когда нужно вычислить средний балл всех студентов в группе. Используя отдельные переменные:

```
const int numStudents = 5;
int student0 = 73;
int student1 = 85;
int student2 = 84;
int student3 = 44;
int student4 = 78;

int totalScore = student0 + student1 + student2 + student3 + student4;
double averageScore = static_cast<double>(totalScore) / numStudents;
```

Мы получим много объявлений переменных и, следовательно, много кода — а это всего лишь 5 студентов! А представьте, если бы не было 30 или 150.

Кроме того, чтобы добавить нового студента, нам придется объявить новую переменную, инициализировать её и добавить в переменную `totalScore` . И это всё вручную. А каждый раз при изменении старого кода есть риск надеять новые ошибок. А вот с использованием массива:

```
const int numStudents = 5;
int students[numStudents] = { 73, 85, 84, 44, 78};
int totalScore = students[0] + students[1] + students[2] + students[3] + students[4];
double averageScore = static_cast<double>(totalScore) / numStudents;
```

Количество объявленных переменных сократится, но в `totalScore` по-прежнему придется заносить каждый элемент массива вручную. И, как указано выше, изменение количества студентов означает, что формулу `totalScore` необходимо будет изменять также вручную.

Если бы был только способ автоматизировать этот процесс.

## Циклы и массивы

Из предыдущего урока мы уже знаем, что индекс массива не обязательно должен быть константным значением — он может быть и переменной. Это означает, что мы можем использовать счетчик цикла в качестве индекса массива для доступа к элементам и выполнения с ними необходимых математических и других операций. Это настолько распространенная практика, что почти всегда при обнаружении массива, вы найдете рядом с ним цикл! Когда цикл используется для доступа к каждому элементу массива поочередно, то это называется **итерацией по массиву**. Например:

```
int students[] = { 73, 85, 84, 44, 78};
const int numStudents = sizeof(students) / sizeof(students[0]);
int totalScore = 0;

// Вычисляем сумму всех значений totalScore
for (int person = 0; person < numStudents; ++person)
    totalScore += students[person];

double averageScore = static_cast<double>(totalScore) / numStudents;
```

Это решение идеально подходит как в плане удобства и чтения, так и поддержки. Поскольку доступ к каждому элементу массива выполняется через цикл, то формула подсчета суммы всех значений автоматически настраивается с учетом количества элементов в массиве. И для вычисления средней оценки нам уже не нужно будет вручную добавлять новых студентов и индексы новых элементов массива!

А вот пример использования цикла для поиска в массиве наибольшего значения (наилучшей оценки среди всех студентов в группе):

```
#include <iostream>

int main()
{

    int students[] = { 73, 85, 84, 44, 78};
    const int numStudents = sizeof(students) / sizeof(students[0]);

    int maxScore = 0; // отслеживаем самую высокую оценку
    for (int person = 0; person < numStudents; ++person)
        if (students[person] > maxScore)
            maxScore = students[person];

    std::cout << "The best score was " << maxScore << "\n";

    return 0;
}
```

Здесь уже используется переменная `maxScore` (не из цикла) для отслеживания самого большого значения массива. Сначала инициализируем `maxScore` значением `0` , что означает, что мы еще не видели никаких оценок. Затем перебираем каждый элемент массива и, если находим оценку, которая выше предыдущей, присваиваем её значение переменной `maxScore` . Таким образом, `maxScore` всегда будет хранить наибольшее значение из всех элементов массива.

## Использование циклов с массивами

Циклы с массивами обычно используются для выполнения одной из трех следующих задач:

- Вычислить значение (например, среднее или сумму всех значений).
- Найти значение (например, самое большое или самое маленькое).
- Отсортировать элементы массива (например, по возрастанию или по убыванию).

При вычислении значения, переменная обычно используется для хранения промежуточного результата, который необходим для вычисления конечного значения. В примере, приведенном выше, где мы вычисляем средний балл, переменная `totalScore` содержит сумму значений всех рассмотренных элементов.

При поиске значения, переменная обычно используется для хранения наилучшего варианта (или индекса наилучшего варианта) из всех просмотренных. В примере, приведенном выше, где мы используем цикл для поиска наивысшей оценки, переменная `maxScore` используется для хранения наибольшего количества баллов из просмотренных ранее элементов массива.

Сортировка массива происходит несколько сложнее, так как в этом деле используются вложенные циклы (но об этом уже на следующем уроке).

## Массивы и «ошибка неучтенной единицы»

Одной из самых сложных задач при использовании циклов с массивами является убедиться, что цикл выполняется правильное количество раз. **Ошибку на единицу** (или **«ошибку неучтенной единицы»**) сделать легко, а попытка получить доступ к элементу, индекс которого больше, чем длина массива, может иметь самые разные последствия. Рассмотрим следующую программу:

```
#include <iostream>

int main()
{

    int students[] = { 73, 85, 84, 44, 78 };
    const int numStudents = sizeof(students) / sizeof(students[0]);

    int maxScore = 0; // отслеживаем самую высокую оценку
    for (int person = 0; person <= numStudents; ++person)
        if (students[person] > maxScore)
            maxScore = students[person];

    std::cout << "The best score was " << maxScore << "\n";

    return 0;
}
```

Здесь проблема состоит в неверном условии **оператора if** в цикле for! Объявленный массив содержит 5 элементов, пронумерованных от 0 до 4. Однако цикл внутри перебирает элементы от 0 до 5. Следовательно, на последней итерации в цикле for выполняется:

```
    if (students[5] > maxScore)
        maxScore = students[5];
```

Но ведь `students[5]` не определен! Его значением, скорее всего, будет простой мусор. И в итоге результатом выполнения цикла может быть ошибочный `maxScore` .

Однако представьте, что бы произошло, если бы мы ненароком присвоили значение элементу `students[5]` ! Мы бы могли перезаписать другую переменную (или её часть) или испортить что-либо — эти типы ошибок очень трудно отследить!

Следовательно, при использовании циклов с массивами, всегда перепроверяйте условия в циклах, чтобы убедиться, что их выполнение не приведет к ошибке неучтенной единицы.

## Тест

### Задание №1

Выведите на экран следующий массив с помощью цикла:

```
int array[] = { 7, 5, 6, 4, 9, 8, 2, 1, 3 };
```

> **Подсказка:** Используйте трюк с sizeof (из предыдущего урока) для определения длины массива.

### Задание №2

Используя массив из задания №1:

Попросите пользователя ввести число от 1 до 9. Если пользователь введет что-либо другое — попросите его снова ввести число и так до тех пор, пока он не введет корректное значение из заданного диапазона. Как только пользователь введет число от 1 до 9, выведите массив на экран. Затем найдите в массиве элемент с числом, которое ввел пользователь, и выведите его индекс.

Для **обработки некорректного ввода** используйте следующий код:

```
std::cin >> number; // Пытаемся ввести некорректное значение
if (std::cin.fail())
{
    std::cin.clear();
    std::cin.ignore(32767, '\n');
}
```

### Задание №3

Измените следующую программу так, чтобы вместо `maxScore` с наибольшим значением, переменная `maxIndex` содержала индекс элемента с наибольшим значением:

```
#include <iostream>

int main()
{

    int scores[] = { 73, 85, 84, 44, 78 };
    const int numStudents = sizeof(scores) / sizeof(scores[0]);

    int maxScore = 0; // отслеживаем самую высокую оценку
    for (int student = 0; student < numStudents; ++student)
        if (scores[student] > maxScore)
            maxScore = scores[student];

    std::cout << "The best score was " << maxScore << "\n";

    return 0;
}
```

## Ответ №1

```
#include <iostream>

int main()
{
    int array[] = { 7, 5, 6, 4, 9, 8, 2, 1, 3 };
    const int arrayLength = sizeof(array) / sizeof(array[0]);

    for (int index=0; index < arrayLength; ++index)
        std::cout << array[index] << " ";
    return 0;
}
```

### Ответ №2

```
#include <iostream>

int main()
{
    // Сначала принимаем корректный пользовательский ввод
    int number = 0;
    do
    {
        std::cout << "Enter a number between 1 and 9: ";
        std::cin >> number;

        // Если пользователь ввел некорректное значение
        if (std::cin.fail())
            std::cin.clear();

        std::cin.ignore(32767, '\n');

    } while (number < 1 || number > 9);

    // Выводим массивом массив на экран
    int array[] = { 7, 5, 6, 4, 9, 8, 2, 1, 3 };
    const int arrayLength = sizeof(array) / sizeof(array[0]);

    for (int index=0; index < arrayLength; ++index)
        std::cout << array[index] << " ";

    std::cout << "\n";

    // Затем ищем в массиве число, которое ввел пользователь и выводим его индекс
    for (int index=0; index < arrayLength; ++index)
    {
        if (array[index] == number)
        {
            std::cout << "The number " << number << " has index " << index << "\n";
            break; // так как мы нашли элемент в массиве уникальный, то нет надобности продолжать перебирать элементы дальше
        }
    }

    return 0;
}
```

### Ответ №3

```
#include <iostream>

int main()
{
    int scores[] = { 73, 85, 84, 44, 78 };
    const int numStudents = sizeof(scores) / sizeof(scores[0]);

    int maxIndex = 0; // отслеживаем самую высокую оценку

    for (int student = 0; student < numStudents; ++student)
        if (scores[student] > scores[maxIndex])
            maxIndex = student;

    std::cout << "The best score: " << scores[maxIndex] << "\n";
    std::cout << "Index of the best score: " << maxIndex << "\n";

    return 0;
}
```